REQUIREMENTS NOT MET

N/A

PROBLEMS ENCOUNTERED

N/A

FUTURE WORK/APPLICATIONS

THIS LAB TEACHES ABOUT HARDWARE TIMERS WHICH WILL BE USED IN FUTURE SOFTWARE AND INTERRUPTS

PRE-LAB EXERCISES

i. Which configuration register allows the utilization of a subset of I/O port pins to be configured as inputs (without affecting other pins)? Which configuration register allowed the utilization of a subset of I/O port pins to be configured as an output (without affecting other pins)?

DIR (data Direction register) is use to sets the data direction for pins of the port. If DIRn is written to 1, pin n is configured as an output pin. If DIRn is written to 0, pin n is configured as input pin.

ii. What is the purpose of the SET/CLR/TGL variants of the DIR and OUT registers?

DIRSET is use to set individual pins as output. DIRCLR will set individual pins as input pin.

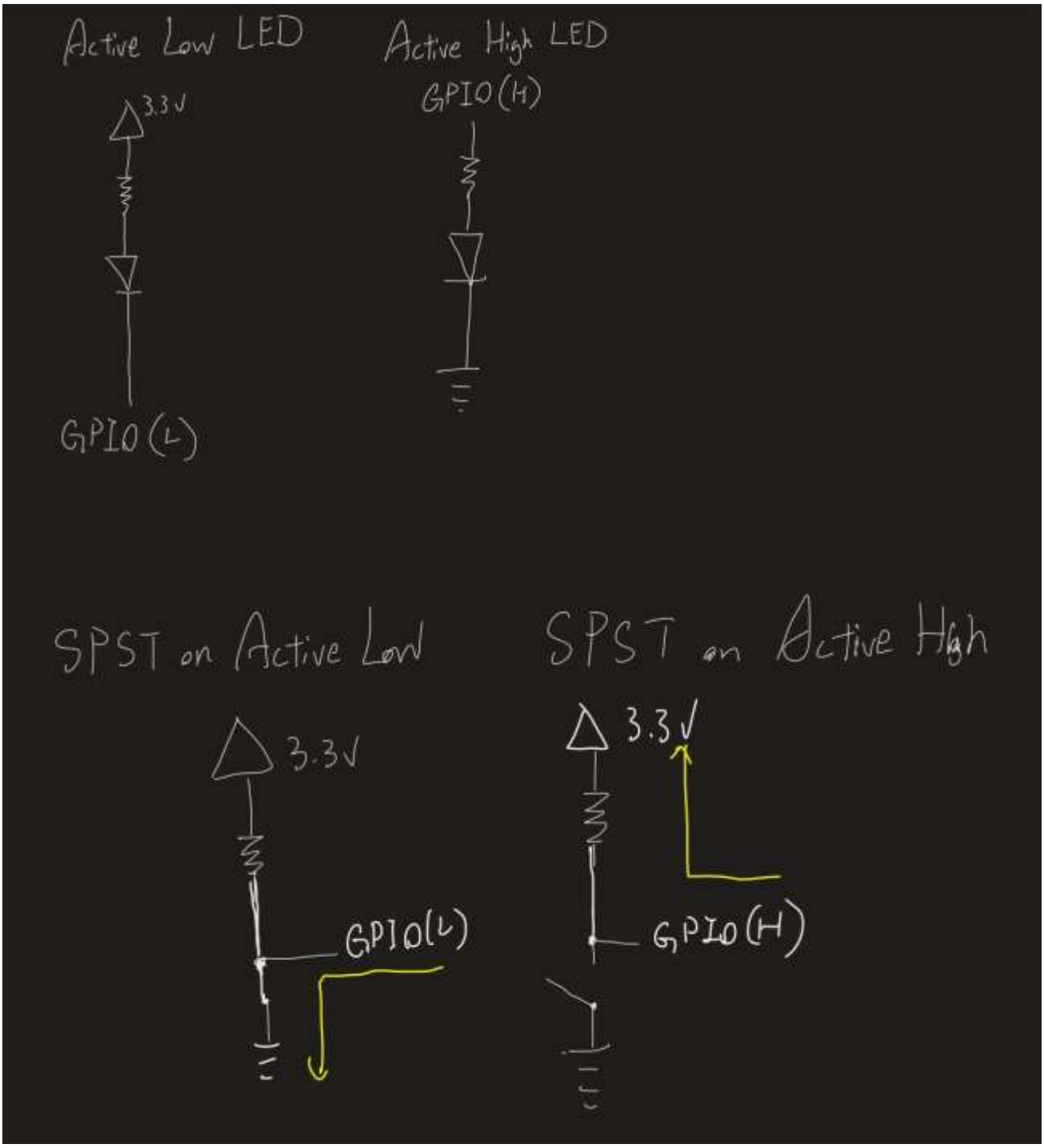
DIRCLR register is use to set the pin as input. DIRSET is use to set individual pin as output.

DIRTGL is use to toggle the status of the individual pins.

iii. Are the LEDs on the OOTB Switch & LED Backpack active-high, or active-low?

The LEDs are Active Low because the Cathodes side (-) is connected to the GND, and the Anode side (+) is connected to the 3.3V.

Draw a schematic diagram for a single LED circuit with the same activation level used on the backpack, as well as one with the opposite activation level. Also, draw a schematic diagram for a single-pole, single-throw (SPST) switch circuit, using the same pull-up or pull-down resistor condition utilized on the backpack, as well as another switch circuit using the opposite configuration.



iv. Which I/O ports are utilized for the DIP switches and LEDs on the OOTB Switch & LED Backpack?

Ports PA0-PA7 are utilized for the DIP switches and PC0-PC7 are untilized for the LEDs.

v. Would it be possible to interface the OOTB µPAD with an external input device consisting of 22 inputs? If so, describe how many I/O ports would be necessary. If not, explain why

It would be possible to interface, because 8 I/O ports is necessary, since each of them has 3 pins.

vi. Assuming a system clock frequency of 2 MHz, a prescaler value of 256, and a desired timer/clock period of 55 ms, calculate a theoretically-corresponding timer/counter period value two separate times: once using a form of dimensional analysis, providing explanation(s) when appropriate, and another time using the general formula provided within The Most Common Use Case for Timer/Counters.

System CIK : 2MHz, Prescaler N : 256, desired Time/CLK 55ms = 55000us

f_timer = f_clk / N = 7812.5Hz

timer period per tick T_tick = 1/7812.5 = 128 us

timer/counter period value : 55000/128 = 429.7 ticks

General Formula:

PER = T * fclk / N – 1= 428.7

vii. Assuming a system clock frequency of 2 MHz, is a period of two seconds achievable when using a 16-bit timer/counter prescaler value of one? If not, determine if there exists any prescaler value that allows for this period under the assumed circumstances, and if there does, list such a value.

f_clk = 2MHz

Timer is 16 bits, maximum it can count is 2^16 = 65536

Time_max = 65536/ (f_clk/N) = 65536/(2000000/1) = 0.0328s = 32.8 ms

Therefore, it can't count for 2 seconds with prescaler 1.

viii. What is the maximum time value (to the nearest millisecond) representable by a timer/counter, if the relevant system clock frequency is 2 MHz? What about for a system clock frequency of 37.444 kHz?

The largest prescaler is 1024

T_max= 65536 * N/ f_clk = 65536 * 1024 / 2000000 = 33.55s = 33550 ms

For f_clk = 37.444kHz,

T_max = 65536 * 1024 / 37444 = 1792s = 1792000ms

ix. Create an assembly program to perform the same procedure as in § 3.2 but utilize a prescaler value of 8. Perform everything else described in the section for this new context, i.e., experimentally determine which whole number digital period value provides a corresponding period with the least amount of error, provide an appropriate screenshot of the relevant waveform with the minimal amount of error, including its precise frequency, and provide within the caption of the relevant screenshot the whole-number value that resulted in a minimal amount of error. Finally, describe and explain why there may be any differences between the two contexts, i.e., between using a prescaler value of 256, the value in exercise vi, and a prescaler value of 8.

The difference between prescaler 8 and 256 is that using a smaller prescaler 8 gives a higher timer resolution and more precise timing with larger PER values, while a larger prescaler 256 produces timing with smaller PER values and higher rounding error.

x. Create an assembly program to keep track of elapsing minutes with a timer/counter, i.e., design a "watch" that only has a "minute-hand". (Hint: Instead of attempting to configure the period of the timer/counter to directly correspond to sixty seconds, configure the period to correspond to one second, and then keep track of how many times this timer/counter overflows [or underflows, if you wish to configure the timer/counter to count down].)

Configure the timer to overflow every 1 second, incrementing a software seconds counter on each overflow, and then incrementing a minutes counter every time the seconds counter reaches 60.

xi. It is stated above that, in the relevant context, it should not be necessary to debounce (nor wait for the release of) either tactile switch S2 on the OOTB MB or S1 on the SLB. Why is this so?

Debouncing is unnecessary for the PLAY and STOP switches because they trigger a single, non-repeating state change, making any subsequent signals from bouncing irrelevant when new mode is entered.

xii. Provide a scenario in which the above program would experience unintended behavior due to tactile switch bouncing.

If the "Store Frame" switch were not debounced, a single physical press could bounce and be registered as multiple presses, causing the same frame to be unintentionally saved several times in the animation.

PSEUDOCODE/FLOWCHARTS

Part 1

Main:

Initialized the Stack

Initialize necessary ports for Switches/LEDs

Start the infinity loop to constantly detect if switch S1 is pressed

Loop:

Read the status of Input, or Switch S1

If Not pressed, {
jump back to loop

}

Part 2

Initialize stack

CPU_SP point to highest data memory 0x3FFF

//void
Delay_10_ms:

// 10ms = 10000us

// 10000us/(0.5us per cycle) = 20000 cycles in order to delay for 10ms

Ldi r17, 100

Ldi r16, 50

```
Outer loop run for 50 times{
        Nop; 1cycle
Inner loop for 100 times {
        Nop ;  1 cycle
        Dec  r17  ; 1cycle
        Brne inner  ; 2 cycle
}
}

// input : r20  (number of 10ms delays)
Delay_x_10ms:
        If (r20 == 0 ){
        Branch to Exit  ; 2 cycles
}
        Rcall Delays_10ms  ; 1cyles
        Dec r20  ;  1 cycles
        Rjmp delayloop

Exit:




Main:
   r16 <- 1                    ; binary 0000 0001
   PORTC_DIR <- r16            ; set bit0 of PORTC as output
```

ToggleLoop:

r16 <- PORTC_OUTTGL ; load toggle register value

r16 <- r16 OR 1 ; ensure bit0 is 1 (others unchanged)

PORTC_OUTTGL <- r16 ; writing 1 toggles PORTC bit0

Call delay_x_10ms with parameter 4

Jump to ToggleLoop

PROGRAM CODE

```
/*
 * Lab2 part2_ 10ms delay , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: This main routine control the LED base on switches
 *
 */

.include "ATxmega128a1udef.inc"

; Replace with your application code
```

```
.cseg
.org 0x0
rjmp main

.org 0x0100
main:


    ;initalize all bits in LEDs to be 1. which the light will turn off
    ldi r16, 0xFF    ; load 0b1111  into r16
    ; PORTC is LEDs
    sts PORTC_OUT, r16
    ; set dirction of all LED pins to outputs
    sts PORTC_DIR, r16

    ;initalize Switch pins (portA) as inputs
    ;load all bits of PORTA as 0
    ldi r16, 0x00
    sts PORTA_DIR, r16


    ;start the loop to constantly checking if any button are pressed
Loop:
    ;use lds to load from a memory address in SRAM
    ;read the DIP switch states into r16
```

```
    lds r16, PORTA_IN
    ; store direct to SRAM/IO
    ; each bit in r16 sets the corresponding pin to 1 or 0
    sts PORTC_OUT, r16 ;changes the current output value for the LEDs

    rjmp Loop
```

```
/*
 * Lab2 part2_ 10ms delay , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: This main routine toggle LED bit0 every 10ms
 *
 */

    ; Stack pointer is a CPU register
    ;
.include "ATxmega128a1udef.inc"
```

```
.cseg
.org 0x0000
rjmp main



; subroutine to delay 10 miliseconds
;inner loop runs 100 times with 4 instrction cycles . 100 * 4 = 400 cycles
; outter loop runs 50 times with 4 instrctions . total 50 * 4  + 400 * 50 = 20200 cycles
; each cycle on this 2MHZ device is 0.5 microsecond , 20200 * 0.5microsecond * 1ms/1000us = 10.1 milisecond


DELAY_10MS:
    ; since we use r16, push the value onto stack
    push r16
    push r17

    ; number of outer loop
    ldi r16, 50
Outter:
    ;number of inner loop
    ldi r17, 100 ; 1 cycle

Inner:
    nop ; 1 cyc dummy instruction to watse time
```

```
    dec r17 ; 1cyc substract 1 from r17
            ; set Z flag to 1 if result after decrement is 0
    brne Inner ; 2 cycle (branch back to inner as long as r17 not
equal 0 )
                ; if it branches it will need to update PC to branch
target (extra cycle)
dec r16       ; 1 cyc
brne Outter  ; 2 cyc

    ; Removes the value from top of the stack
    ; restore into sp
    ; sp is back to it's previous position

    pop r16
    pop r17

    ret               ; pop return address from stack and jump back

; input : r20 (numbers of 10ms delay)

Delay_x_10ms:
    push r20
Loop_x_times:
    ; save the parameter register o

    tst r20 ; if r20 == 0 , mark z flash to 1,
    breq exit ; brach to exit if r20 == 0
```

```
    rcall DELAY_10MS ; call the delay 10 ms function
    dec r20
    rjmp Loop_x_times


exit:
    pop r20
    ret


.cseg
.org 0x100
; toggle bit0 every 0.04s = 40ms
main:
    ; initialize Stack Pointer at highest Data memory address in the
main
    ; which is 0x3FFF
    ldi r16, 0xFF
    sts CPU_SPL, r16
    ldi r16, 0x3F
    sts CPU_SPH, r16

    ; r16 <- 0000 0001
    ldi r16, 0x01
    ;set bit0 as output
    sts PORTC_DIR, r16


toggle_10ms:
    ; r16 <- 0000 0001
```

```
    ldi r16, 0x01

    sts PORTC_OUTTGL, r16 ; store back to sram
                          ; writing 1 toggles corresponding bit
    ; delay 10ms

    rcall DELAY_10MS ; call function to delay 4 * 10ms
    ; after 40ms jump back to toggle the bit0
    rjmp toggle_10ms
```

```
/*
 * Lab2 part2_40ms delay , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: This main routine toggle LED bit0 every 40ms
 *
 */


    ; Stack pointer is a CPU register
    ;
.include "ATxmega128a1udef.inc"
```

```
.cseg
.org 0x0000
rjmp main



; subroutine to delay 10 miliseconds
;inner loop runs 100 times with 4 instrction cycles . 100 * 4 = 400 cycles
; outter loop runs 50 times with 4 instrctions . total 50 * 4  + 400 * 50 = 20200 cycles
; each cycle on this 2MHZ device is 0.5 microsecond , 20200 * 0.5microsecond * 1ms/1000us = 10.1 milisecond

DELAY_10MS:
    ; since we use r16, push the value onto stack
    push r16

    ; number of outer loop
    ldi r16, 50
Outter:
    ;number of inner loop
    ldi r17, 100 ; 1 cycle

Inner:
    nop ; 1 cyc dummy instruction to watse time
```

```
    dec r17 ; 1cyc substract 1 from r17
            ; set Z flag to 1 if result after decrement is 0
    brne Inner ; 2 cycle (branch back to inner as long as r17 not
equal 0 )
                ; if it branches it will need to update PC to branch
target (extra cycle)
dec r16       ; 1 cyc
brne Outter  ; 2 cyc

    ; Removes the value from top of the stack
    ; restore into sp
    ; sp is back to it's previous position

    pop r16

    ret               ; pop return address from stack and jump back

; input : r20 (numbers of 10ms delay)

Delay_x_10ms:
    push r20
Loop_x_times:
    ; save the parameter register o

    tst r20 ; if r20 == 0 , mark z flash to 1,
    breq exit ; brach to exit if r20 == 0
    rcall DELAY_10MS ; call the delay 10 ms function
```

```
    dec r20
    rjmp Loop_x_times

exit:
    pop r20
    ret

.cseg
.org 0x100
; toggle bit0 every 0.04s = 40ms
main:
    ; initialize Stack Pointer at highest Data memory address in the
main
    ; which is 0x3FFF
    ldi r16, 0xFF
    sts CPU_SPL, r16
    ldi r16, 0x3F
    sts CPU_SPH, r16

    ; r16 <- 0000 0001
    ldi r16, 0x01
    ;set bit0 as output
    sts PORTC_DIR, r16

toggle:
    ; r16 <- 0000 0001
    ldi r16, 0x01
```

```
    sts PORTC_OUTTGL, r16 ; store back to sram
                          ; writing 1 toggles corresponding bit
    ; delay 40ms
    ldi r20, 4  ; load r20 with 4 as input
    rcall Delay_x_10ms ; call function to delay 4 * 10ms
    ; after 40ms jump back to toggle the bit0
    rjmp toggle
```

```
/*
 * Lab2 part3_prescaler 256 , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: toggle every 50ms, perscaler 256
 *
 */
.include "ATxmega128a1udef.inc"

.cseg
.org 0x0000
    rjmp main
```

```
.org 0x0100
main:
; initailze the stack pointer
    ldi r16, low(RAMEND)
    out CPU_SPL, r16
    ldi r16, high(RAMEND)
    out CPU_SPH, r16

;set bit0 of PORTC to be output
    ldi r16, 0x01
    sts PORTC_DIR, r16

    ; Load PER (low byte first)
     ; PER controls how long before overflow occurs
     ; f_timer = f_cpu/prescaler
     ; f_timer = 2,000,000/256 = 7812.5 Hz
     ; Number of timer count = f_timer * T = 7812.5 times/ seconds
     ; for 55ms, timer counts for 7812.5 * 0.055 = 429.6875

    ldi r16, low(428)
    sts TCC0_PER, r16
    ldi r16, high(428)
    sts TCC0_PER+1, r16
     ; Timer counts from 0 → PER, sets overflow flag (OVF) in INTFLAGS
```

```
    ;prescaler controls how fast the timer count
    ; Start Timer C0 with prescaler = 256
    ; TC_CLKSEL_DIV256_gc is bitmask 0x06
    ; 0b110 → prescaler 25
    ldi r16, TC_CLKSEL_DIV256_gc
    ; CTRLA select prescaler
    sts TCC0_CTRLA, r16


loop:
    lds r16, TCC0_INTFLAGS        ; Load Timer C0 interrupt flags into
r16
                                   ; Bit 0 = OVF (overflow flag)
                                   ; Other bits may represent other
flags
    sbrc r16, 0                   ; Skip next instruction if bit 0
(OVF) is clear
                                   ; If OVF = 1, the next instruction
executes
                                   ; If OVF = 0, skip next instruction
and continue loop


    rjmp overflow                  ; Jump to overflow handler if OVF =
1


    rjmp loop                      ; If OVF = 0, stay in loop (poll
again)


overflow:
    ldi r17, 0x01                  ; Load bitmask for PC0 (LED pin)
```

```
    sts PORTC_OUTTGL, r17          ; Toggle LED on PC0
                                    ; Writing '1' flips the pin; '0'
does nothing

      ldi r17, 0x01                  ; Bitmask for OVF flag (bit 0)
    sts TCC0_INTFLAGS, r17         ; Clear overflow flag by writing '1'
                                    ; Necessary to detect next overflow

    rjmp loop                       ; Return to main polling loop
```

```
/*
 * Lab2 part3_prescaler 8 , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: toggle every 50ms, perscaler 8
 *
 */
.include "ATxmega128a1udef.inc"

.cseg
.org 0x0000
    rjmp main
```

```
.org 0x0100
main:
; Initialize the stack pointer
    ldi r16, low(RAMEND)
    out CPU_SPL, r16
    ldi r16, high(RAMEND)
    out CPU_SPH, r16



    ldi r16, 0x01
    sts PORTC_DIR, r16

; Load Timer Period Register (PER) for 55 ms with prescaler = 8
    ; f_timer = f_cpu / prescaler = 2,000,000 / 8 = 250,000 Hz
    ; Number of counts = f_timer * T = 250,000 * 0.055 = 13,750
    ; PER = 13,749
    ldi r16, low(13750)       ; Load low byte of PER
    sts TCC0_PER, r16         ; Store low byte into TCC0 PER
    ldi r16, high(13750)      ; Load high byte of PER
    sts TCC0_PER+1, r16       ; Store high byte into TCC0 PER
    ; Timer counts from 0 → PER, sets overflow flag (OVF) in INTFLAGS

; Start Timer C0 with prescaler = 8
    ; TC_CLKSEL_DIV8_gc = 0x02 → selects prescaler 8
    ldi r16, TC_CLKSEL_DIV8_gc
    sts TCC0_CTRLA, r16       ; Load prescaler into Timer C0 CTRLA
```

```
loop:
; Poll Timer Overflow Flag (OVF)
    lds r16, TCC0_INTFLAGS    ; Load Timer C0 interrupt flags into r16
                               ; Bit 0 = OVF
    sbrc r16, 0               ; Skip next instruction if OVF = 0
    rjmp overflow              ; If OVF = 1, jump to overflow handler
    rjmp loop                  ; Otherwise, continue polling

overflow:
; Handle Timer Overflow
    ldi r17, 0x01             ; Bitmask for PC0 (LED pin)
    sts PORTC_OUTTGL, r17     ; Toggle LED on PC0

; Clear Overflow Flag
    ldi r17, 0x01             ; Bitmask for OVF (bit 0)
    sts TCC0_INTFLAGS, r17    ; Clear OVF by writing '1'

    rjmp loop                  ; Return to main polling loop
```

```
/*
 * Lab2 part3_minutes Counter , Section 17b
 * Name: Peter Feng
 * Class #: 11303
  *PI Name: Grant
  Description: toggle every minutues
 *
 */
.include "ATxmega128a1udef.inc"

.cseg
.org 0x0000
    rjmp main

.org 0x0100
main:
; Initialize the stack pointer
    ldi r16, low(RAMEND)
    out CPU_SPL, r16
    ldi r16, high(RAMEND)
    out CPU_SPH, r16
```

```
    ldi r16, 0x01
    sts PORTC_DIR, r16

; Load Timer Period Register (PER) for 1 second overflow
    ; f_timer = f_cpu / prescaler = 2,000,000 / 256 = 7,812.5 Hz
    ; Number of counts for 1 second = f_timer * 1s ≈ 7,812
    ldi r16, low(7812)
    sts TCC0_PER, r16
    ldi r16, high(7812)
    sts TCC0_PER+1, r16

; Start Timer C0 with prescaler = 256
    ldi r16, TC_CLKSEL_DIV256_gc
    sts TCC0_CTRLA, r16       ; Load prescaler into Timer C0 CTRLA

; Initialize software counters
    clr r18                   ; Seconds counter (0–59)
    clr r19                   ; Minutes counter

loop:
; Poll Timer Overflow Flag (OVF)
    lds r16, TCC0_INTFLAGS    ; Load Timer C0 interrupt flags into r16
    sbrc r16, 0               ; Skip next instruction if OVF = 0
    rjmp overflow             ; Jump to overflow handler if OVF = 1
    rjmp loop                 ; Otherwise, continue polling
```

```
overflow:
; Handle 1-second timer overflow
    ldi r17, 0x01
    sts TCC0_INTFLAGS, r17

; Increment seconds counter
    inc r18                     ; r18 = r18 + 1
    cpi r18, 60                 ; Compare seconds with 60
    brne loop                   ; If < 60, return to loop

; 60 seconds elapsed → increment minutes
    clr r18                     ; Reset seconds counter
    inc r19                     ; Increment minutes counter

; Optional: Toggle LED every minute for visual debug
    ldi r17, 0x01
    sts PORTC_OUTTGL, r17       ; Toggle PC0

    rjmp loop                   ; Return to main polling loop
```

```
;***************************************************
;  lab2_4.asm, section 4
;  Name: Peter Feng
;     PI Name : Grant
;  Purpose: To allow LED animations to be created with the OOTB uPAD,
;                OOTB SLB, and OOTB MB.
;
;***************************************************

;*******INCLUDES*************************************
; The inclusion of the following file is REQUIRED for our course, since
; it is intended that you understand concepts regarding how to specify an
; "include file" to an assembler.
.include "ATxmega128a1udef.inc"
;*******END OF INCLUDES*****************************

;*******DEFINED SYMBOLS*****************************
.equ ANIMATION_START_ADDR   =     0x2000 ;useful, but not required
.equ ANIMATION_SIZE              =     8000 ;useful, but not required, up to 8000
frames
.equ STORE_FRAME_SWITCH_BIT = 3
.equ PLAY_SWITCH_BIT = 2
.equ EDIT_SWITCH_BIT = 1
;*******END OF DEFINED SYMBOLS**********************

;*******MEMORY CONSTANTS****************************
; data memory allocation
.dseg
```

```
.org ANIMATION_START_ADDR
ANIMATION:
.byte ANIMATION_SIZE ; allocate for 8000 frames
;*******END OF MEMORY CONSTANTS**********************

;*******MAIN PROGRAM*********************************
.cseg
; upon system reset, jump to main program (instead of executing
; instructions meant for interrupt vectors)
.org 0x0000
     rjmp MAIN

; place the main program somewhere after interrupt vectors (ignore for now)
.org 0x0100          ; >= 0xFD
MAIN:
; initialize the stack pointer
     ldi r16, low(RAMEND)
    out CPU_SPL, r16
    ldi r16, high(RAMEND)
    out CPU_SPH, r16
; initialize relevant I/O modules (switches and LEDs)
     rcall IO_INIT

; initialize (but do not start) the relevant timer/counter module(s)
     rcall TC_INIT

; Initialize the X and Y indices to point to the beginning of the
; animation table. (Although one pointer could be used to both
; store frames and playback the current animation, it is simpler
```

```
; to utilize a separate index for each of these operations.)
; Note: recognize that the animation table is in DATA memory
     ldi XL, low(ANIMATION_START_ADDR) ; X low
    ldi XH, high(ANIMATION_START_ADDR) ; X high
    ldi YL, low(ANIMATION_START_ADDR) ; Y low
    ldi YH, high(ANIMATION_START_ADDR) ; Y high

; begin main program loop
; "EDIT" mode
EDIT:
; Check if it is intended that "PLAY" mode be started, i.e.,
; determine if the relevant switch has been pressed.
     lds r16, PORTF_IN ; check what's in the PORTF_IN S1
    sbrs r16, PLAY_SWITCH_BIT ; if PORTF bit 2 is 1
    rjmp PLAY    ; jump to play mode
; Otherwise, update LEDs with DIP switch state
    lds r19, PORTA_IN ; get what's in port_in into r19
    sts PORTC_OUT, r19     ; update the corrsponding LED

; If it is determined that relevant switch was pressed,
; go to "PLAY" mode.
; Otherwise, if the "PLAY" mode switch was not pressed,
; update display LEDs with the voltage values from relevant DIP switches
; and check if it is intended that a frame be stored in the animation
; (determine if this relevant switch has been pressed).
     lds r16, PORTF_IN          ; check if store is pressed
     sbrc r16, STORE_FRAME_SWITCH_BIT ;
     rjmp EDIT
```

```
; If the "STORE_FRAME" switch was not pressed,
; branch back to "EDIT".
; Otherwise, if it was determined that relevant switch was pressed,
; perform debouncing process, e.g., start relevant timer/counter
; and wait for it to overflow. (Write to CTRLA and loop until
; the OVFIF flag within INTFLAGS is set.)
     ldi r16, 0x06                ; prescaler = 256
     sts TCC1_CTRLA, r16
WAIT_DEBOUNCE:
     lds r17, TCC1_INTFLAGS
     sbrs r17, 0                  ; wait for overflow
     rjmp WAIT_DEBOUNCE
; After relevant timer/counter has overflowed (i.e., after
; the relevant debounce period), disable this timer/counter,
; clear the relevant timer/counter OVFIF flag,
; and then read switch value again to verify that it was
; actually pressed. If so, perform intended functionality, and
; otherwise, do not; however, in both cases, wait for switch to
; be released before jumping back to "EDIT".
     ldi r16, 0x00
     sts TCC1_CTRLA, r16          ; stop timer
     ldi r16, 0x01
     sts TCC1_INTFLAGS, r16       ; clear overflow flag

     lds r16, PORTF_IN
     sbrc r16, STORE_FRAME_SWITCH_BIT
     rjmp EDIT
; ---- Perform intended functionality (store frame) ----
     lds r19, PORTA_IN
```

```
      st X+, r19                    ; save current DIP switch state
                                          ; also incre x so next frame goes to
next slot


; Wait for the "STORE FRAME" switch to be released
; before jumping to "EDIT".
STORE_FRAME_SWITCH_RELEASE_WAIT_LOOP:
      lds r16, PORTF_IN
      sbrs r16, STORE_FRAME_SWITCH_BIT  ; if s1 trun back to 1
                                                ; go to edit mode
                                                ; else, continously checking
      rjmp STORE_FRAME_SWITCH_RELEASE_WAIT_LOOP
      rjmp EDIT


; "PLAY" mode
PLAY:
; Reload the relevant index to the first memory location
; within the animation table to play animation from first frame.
      ldi YL, low(ANIMATION_START_ADDR)
      ldi YH, high(ANIMATION_START_ADDR)


PLAY_LOOP:
; Check if it is intended that "EDIT" mode be started
; i.e., check if the relevant switch has been pressed.`
      lds r16, PORTE_IN
      sbrs r16, EDIT_SWITCH_BIT



; If it is determined that relevant switch was pressed,
; go to "EDIT" mode.
```

```
rjmp EDIT
; Otherwise, if the "EDIT" mode switch was not pressed,
; determine if index used to load frames has the same
; address as the index used to store frames, i.e., if the end
; of the animation has been reached during playback.
; (Placing this check here will allow animations of all sizes,
; including zero, to playback properly.)
; To efficiently determine if these index values are equal,
; a combination of the "CP" and "CPC" instructions is recommended.
     cp YL, XL
     cpc YH, XH ;CPC continues the comparison with the next higher byte
     breq PLAY ; restart if reached end

; If index values are equal, branch back to "PLAY" to
; restart the animation.
; Otherwise, load animation frame from table,
; display this "frame" on the relevant LEDs,
; start relevant timer/counter,
; wait until this timer/counter overflows (to more or less
; achieve the "frame rate"), and then after the overflow,
; stop the timer/counter,
; clear the relevant OVFIF flag,
; and then jump back to "PLAY_LOOP".
     ld r16, Y+
     sts PORTC_OUT, r16

     ldi r16, 0x05               ; prescaler = 64
     sts TCC0_CTRLA, r16
WAIT_FRAME:
```

```
      lds r17, TCC0_INTFLAGS ; wait till overflow (when INTFLAGS is 1)
      sbrs r17, 0
      rjmp WAIT_FRAME
      ldi r16, 0x00
      sts TCC0_CTRLA, r16
      ldi r16, 0x01
      sts TCC0_INTFLAGS, r16
      rjmp PLAY_LOOP

; end of program (never reached)
DONE:
      rjmp DONE
;*******END OF MAIN PROGRAM *************************

;*******SUBROUTINES***********************************
;************************************************
; Name: IO_INIT
; Purpose: To initialize the relevant input/output modules, as pertains to the
;             application.
; Input(s): N/A
; Output: N/A
;************************************************
IO_INIT:
; protect relevant registers
      push r16

; initialize the relevant I/O
      ldi r16, 0xFF            ;
      sts PORTC_DIR, r16       ; LEDs = output
```

```
      sts PORTC_OUT, r16
      sts PORTA_OUT, r16
      clr r16       ; r16 is now 0x00
      sts PORTA_DIR, r16       ; Switches = input

; recover relevant registers
      pop r16

; return from subroutine
      ret

;***************************************************
; Name: TC_INIT
; Purpose: To initialize the relevant timer/counter modules, as pertains to
;             application.
; Input(s): N/A
; Output: N/A
;***************************************************
TC_INIT:
; protect relevant registers
      push r16

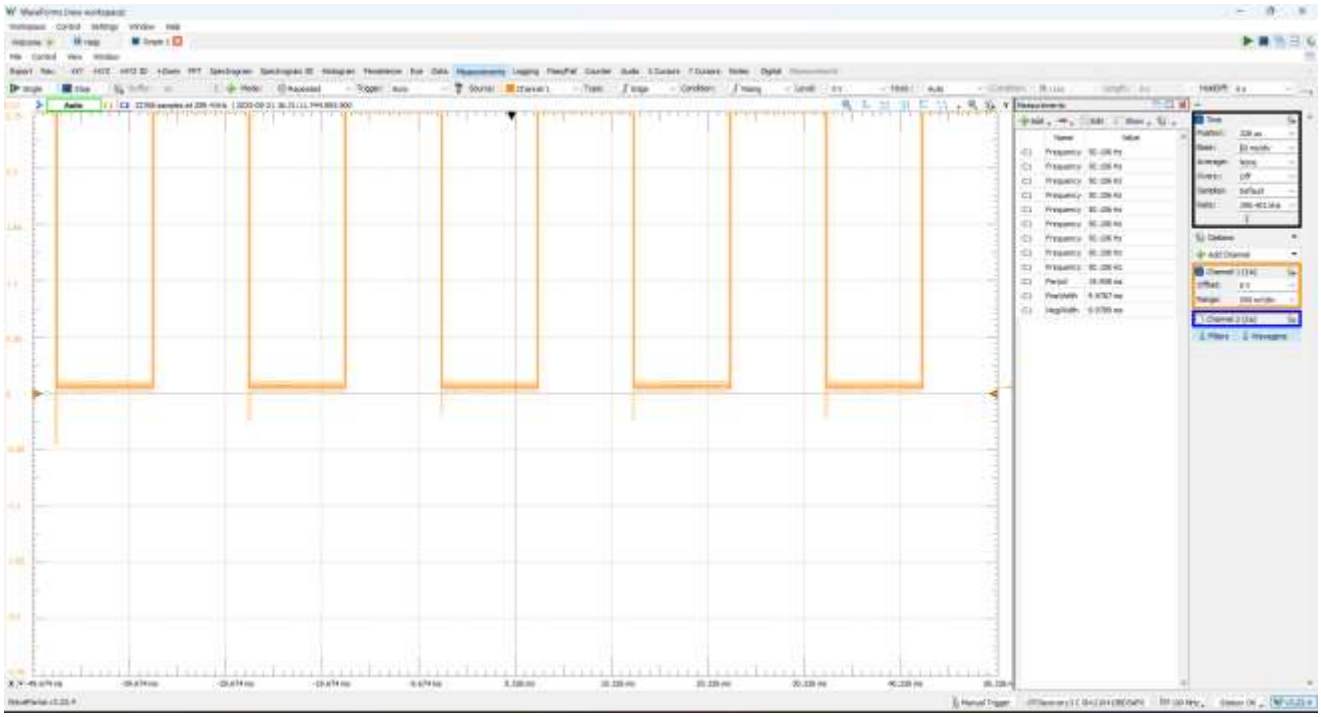
; initialize the relevant TC modules
;framrate timer
      ldi r16, low(7812)       ; PER = 7812 for ~500ms @ prescaler 64
      sts TCC0_PER, r16
      ldi r16, high(7812)
      sts TCC0_PER+1, r16
      ldi r16, 0x00
```

```
      sts TCC0_CTRLA, r16      ; off initially
;hardware timer
      ldi r16, low(150)       ; PER =10ms
      sts TCC1_PER, r16
      ldi r16, high(150)
      sts TCC1_PER+1, r16
      ldi r16, 0x00
      sts TCC1_CTRLA, r16      ; off initially

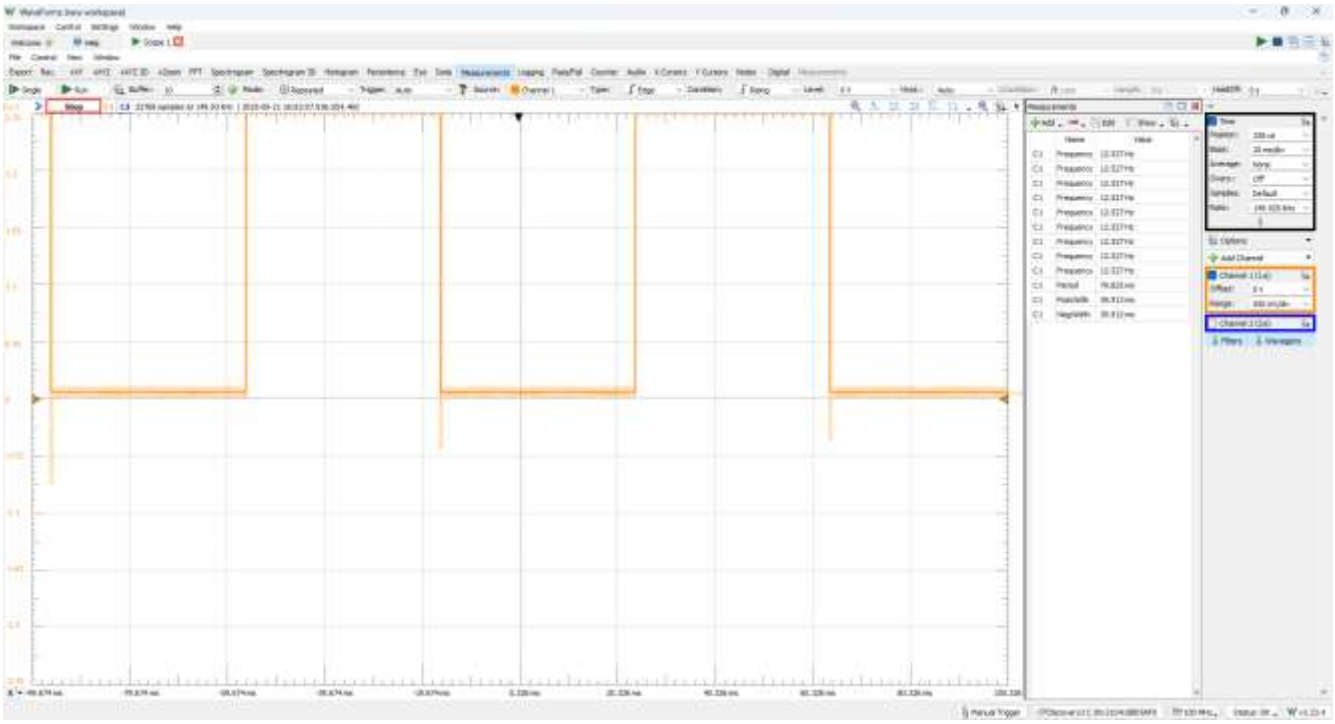
; recover relevant registers
      pop r16

; return from subroutine
      ret
;*******END OF SUBROUTINES**************************
```

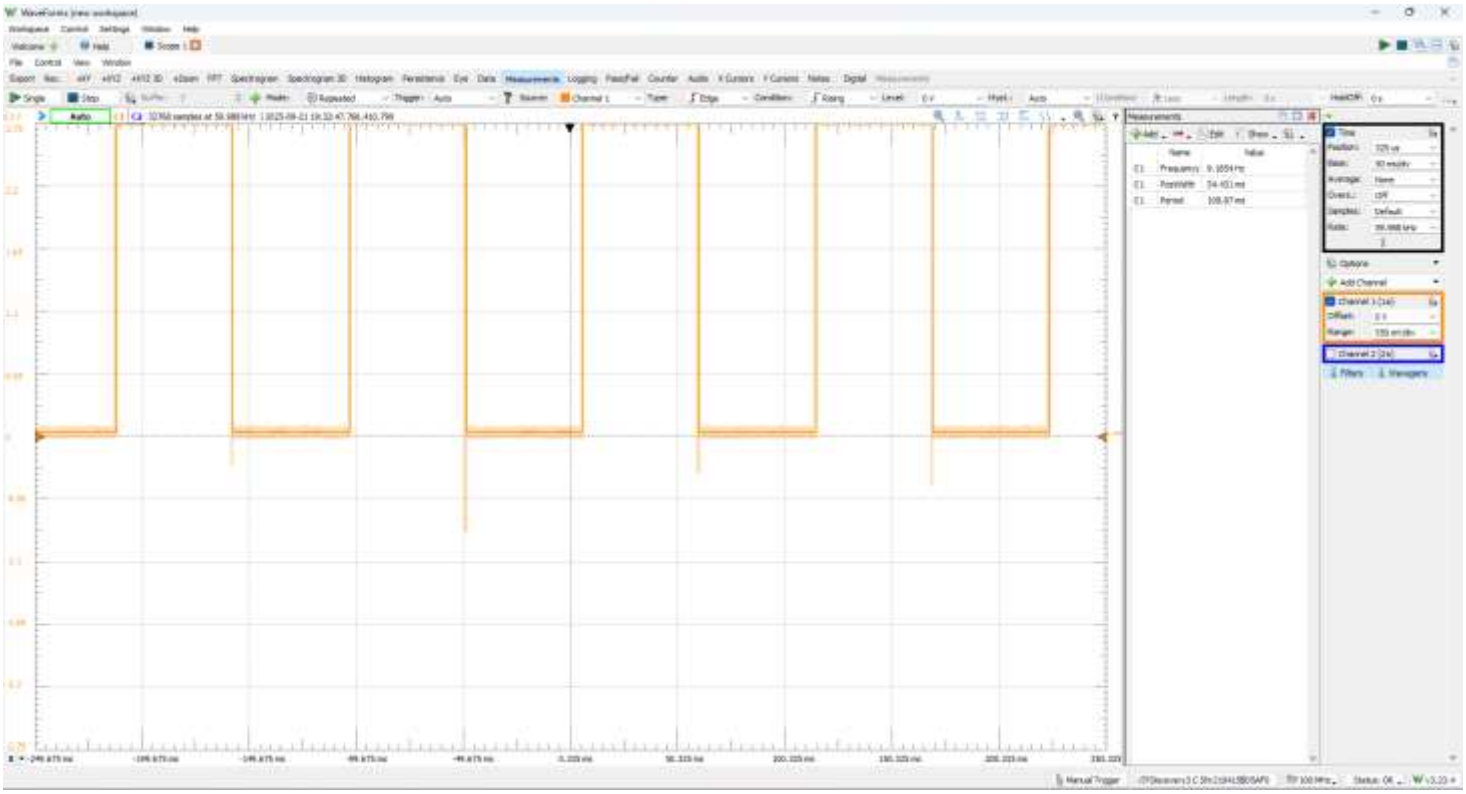
APPENDIX



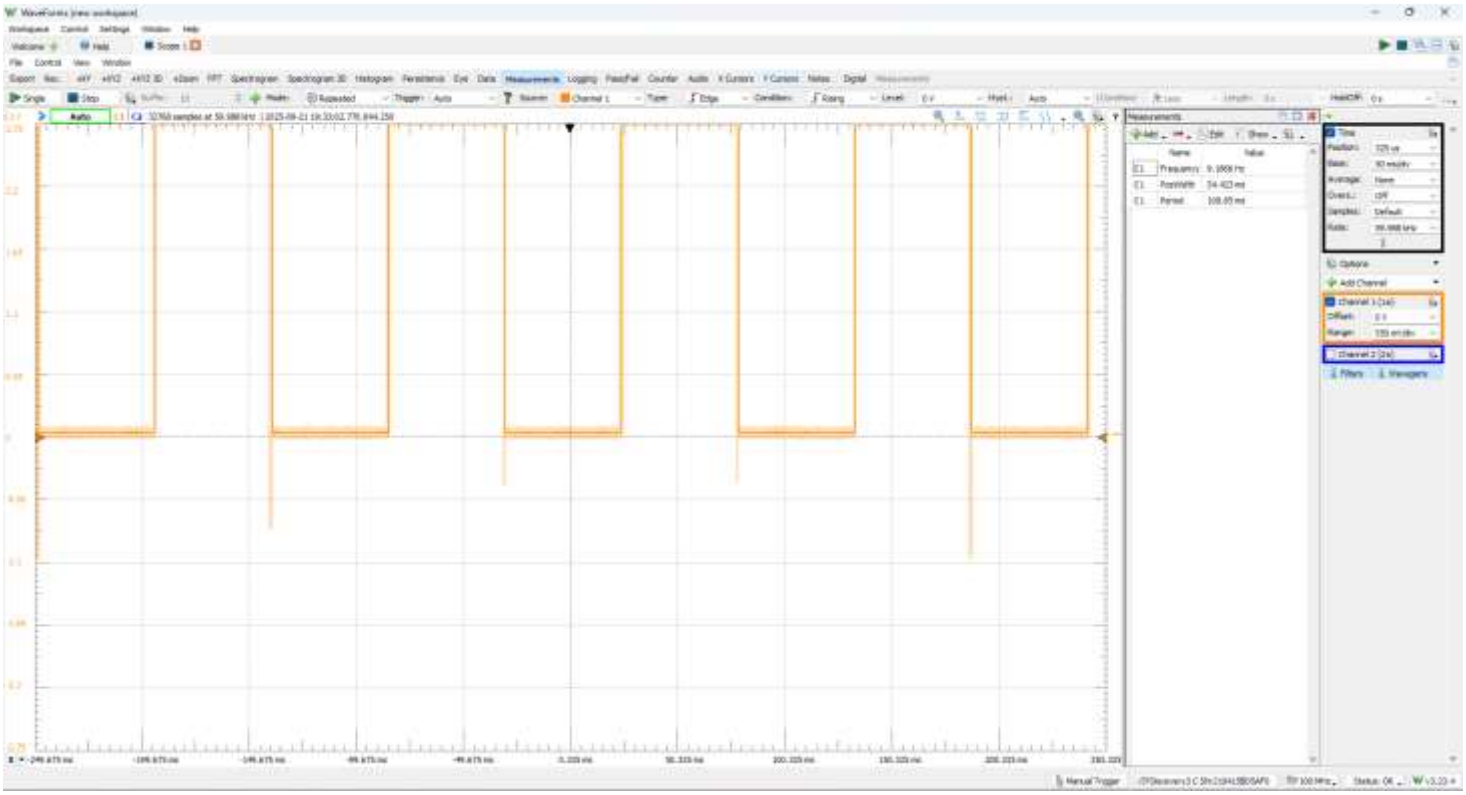
Part 2_ 10ms delay



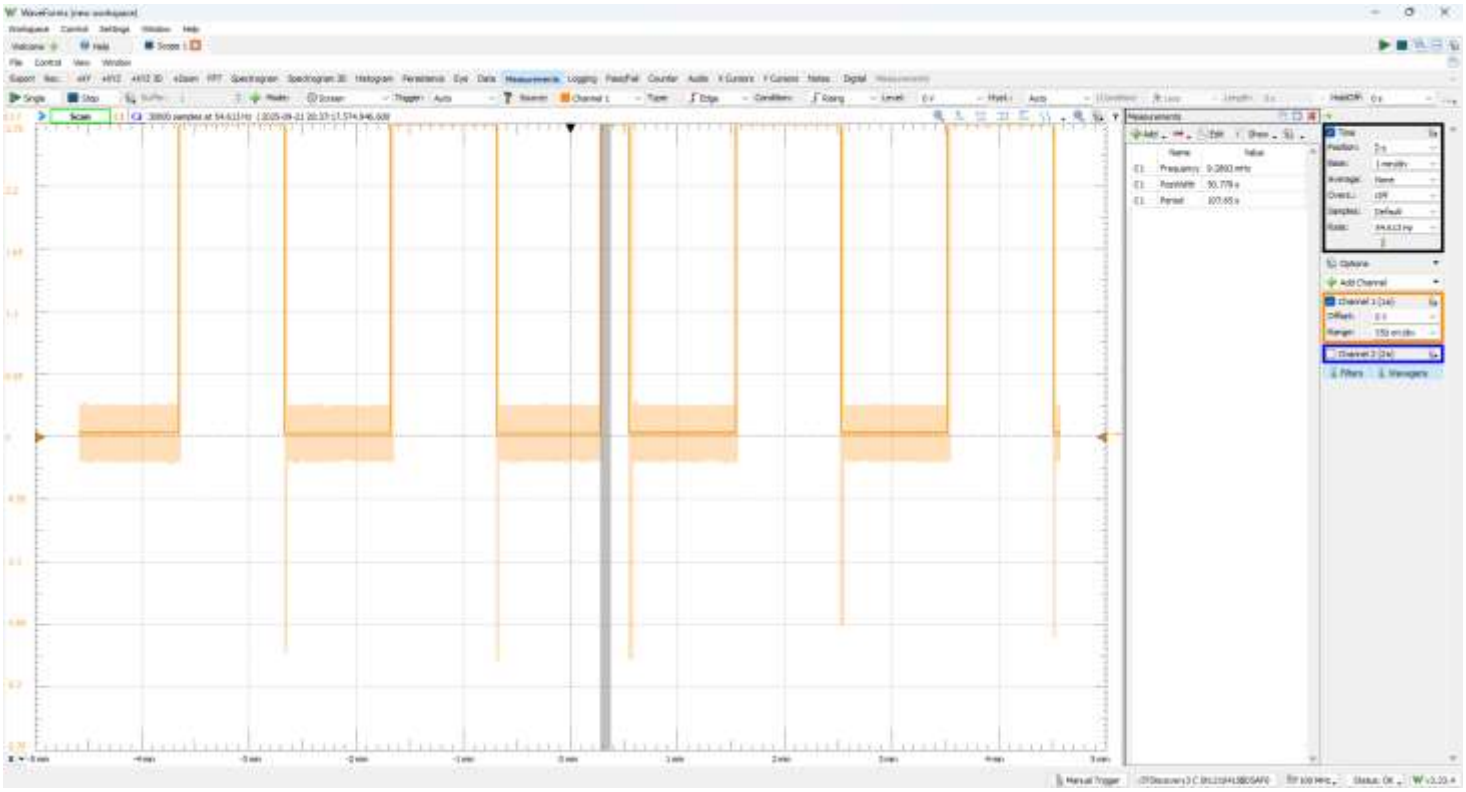
Part2_40ms delay



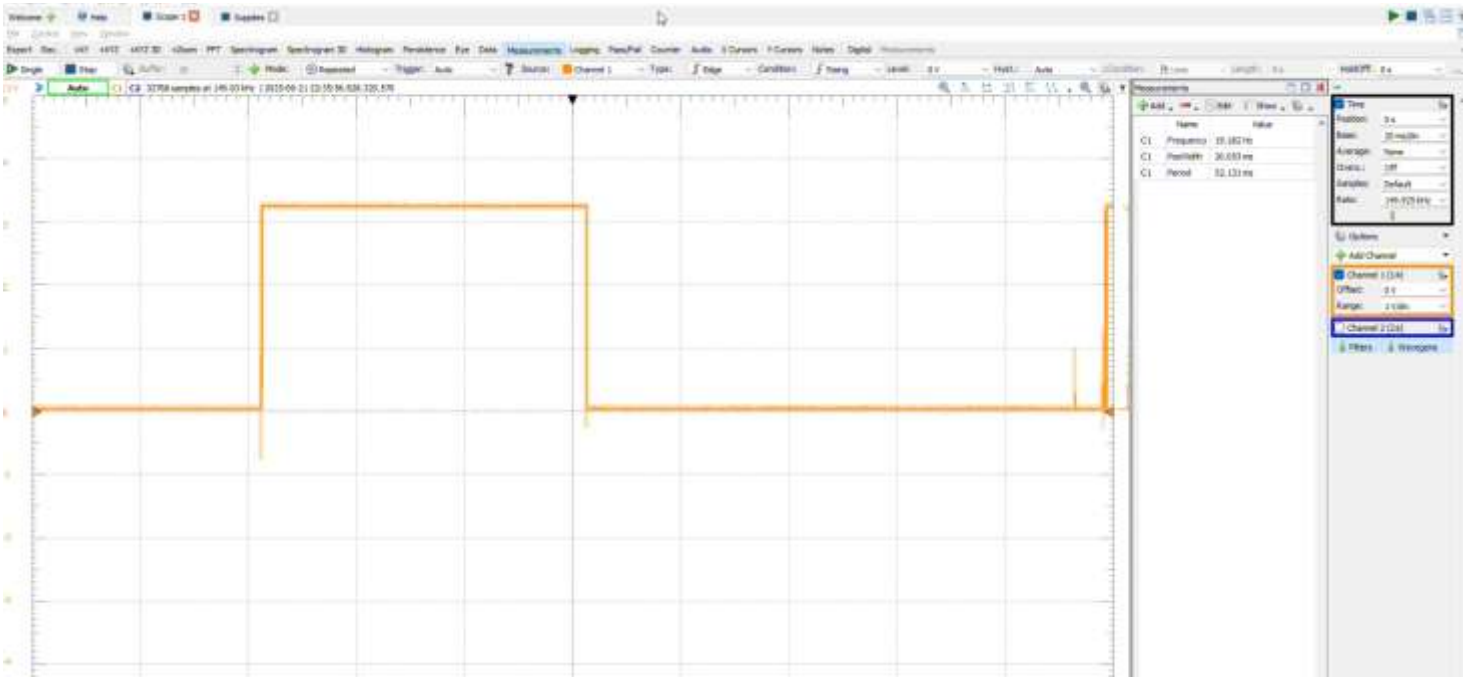
Part3 55ms delay with prescaler 256



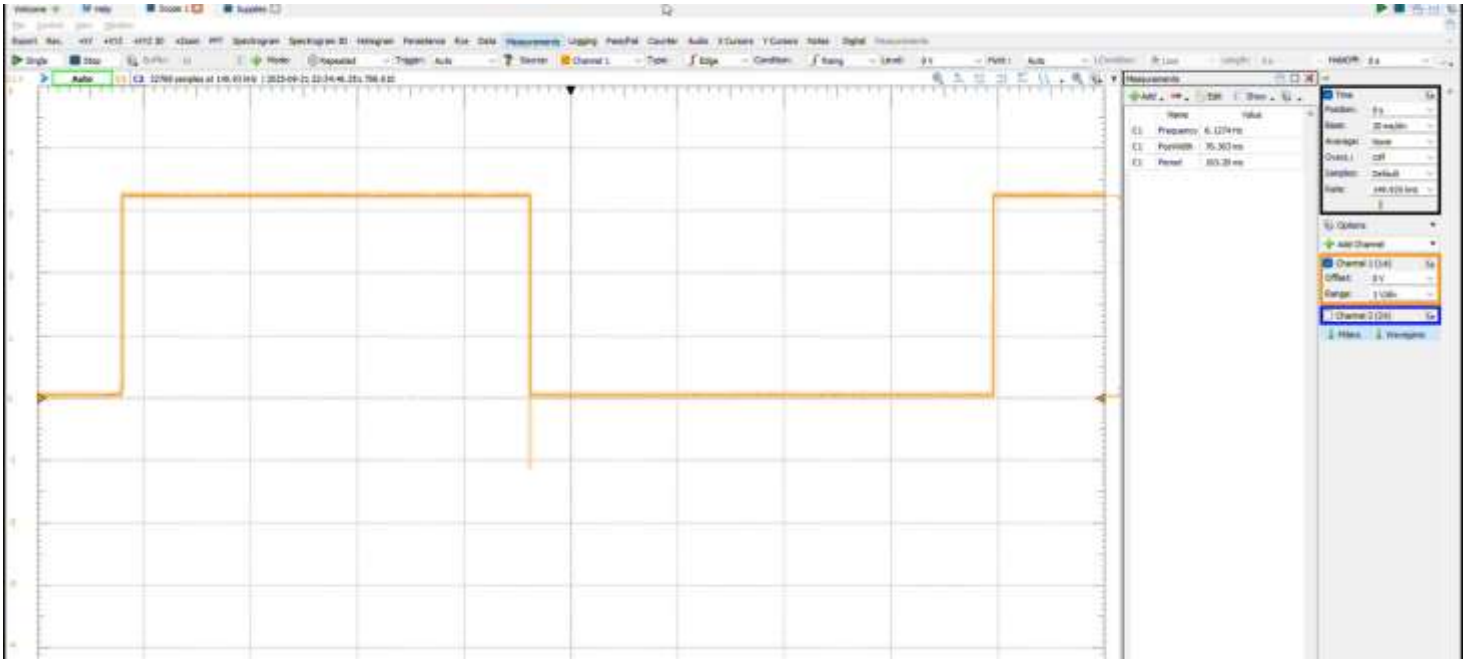
Part3 55ms delay with prescaler 8



Part3 1 minutes delay



Part4 switch 1 debounce



Part4 switch 2 debounce